Module 3: AI for Mechatronics

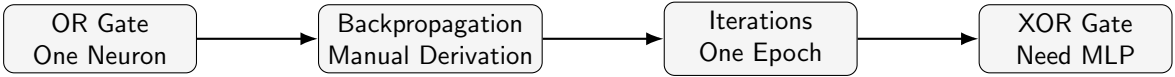
ANN : Back Propagation Derivation

Dr. Rajan Prasad

SMEC
VIT University

Fall 2026

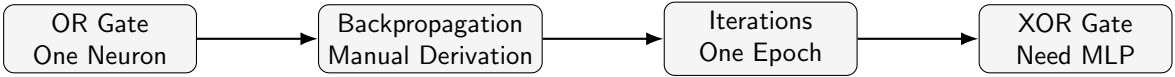
Learning Story for This Lecture



Main question

Can a machine learn a logical rule only from examples?

Learning Story for This Lecture



Main question

Can a machine learn a logical rule only from examples?

We first learn OR by hand, then use XOR to understand why hidden layers are needed.

Can a Neural Network Learn the OR Gate?

Training data

x_1	x_2	Desired output y_d
0	0	0
0	1	1
1	0	1
1	1	1

Question for the class

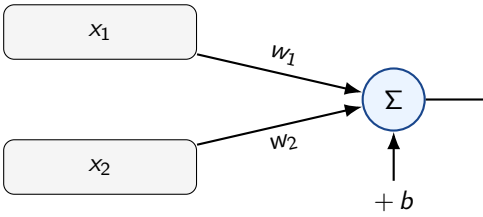
Can the model discover the OR rule?

We do not explicitly program the logical condition. We only provide examples and let the neuron adjust its weights.

- Forward propagation
- Prediction error and loss
- Backpropagation
- Gradient-descent weight update

Goal: follow one complete epoch by hand.

Artificial Neuron for the OR Gate



Forward equations

$$z = w_1 x_1 + w_2 x_2 + b$$

$$y = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Interpretation

The neuron forms a weighted sum and converts it into an output between 0 and 1.

Forward Propagation and Loss

For one training sample,

$$(x_1, x_2) \longrightarrow y_d$$

Forward Propagation and Loss

For one training sample,

$$(x_1, x_2) \longrightarrow y_d$$

Forward pass

$$z = w_1 x_1 + w_2 x_2 + b$$

$$y = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Forward Propagation and Loss

For one training sample,

$$(x_1, x_2) \longrightarrow y_d$$

Forward pass

$$z = w_1 x_1 + w_2 x_2 + b$$

$$y = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Error and loss

$$e = y_d - y$$

$$J = \frac{1}{2}(y_d - y)^2$$

Forward Propagation and Loss

For one training sample,

$$(x_1, x_2) \longrightarrow y_d$$

Forward pass

$$z = w_1 x_1 + w_2 x_2 + b$$

$$y = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Error and loss

$$e = y_d - y$$

$$J = \frac{1}{2}(y_d - y)^2$$

Teaching point

Forward propagation tells us what the neuron predicts. The loss tells us how wrong that prediction is.

Why Do We Need Backpropagation?

After the forward pass, we know the loss:

$$J = \frac{1}{2}(y_d - y)^2$$

Why Do We Need Backpropagation?

After the forward pass, we know the loss:

$$J = \frac{1}{2}(y_d - y)^2$$

But the real question is:

How should $w_1,\ w_2,\ b$ change to reduce J?

Why Do We Need Backpropagation?

After the forward pass, we know the loss:

$$J = \frac{1}{2}(y_d - y)^2$$

But the real question is:

How should $w_1,\ w_2,\ b$ change to reduce J?

Backpropagation calculates the effect of each parameter on the loss.

Why Do We Need Backpropagation?

After the forward pass, we know the loss:

$$J = \frac{1}{2}(y_d - y)^2$$

But the real question is:

How should $w_1,\ w_2,\ b$ change to reduce J?

Backpropagation calculates the effect of each parameter on the loss.

For each parameter, we need gradients:

$$\frac{\partial J}{\partial w_1}, \qquad \frac{\partial J}{\partial w_2}, \qquad \frac{\partial J}{\partial b}$$

Chain Rule for One Neuron

The loss depends on weights indirectly:

$$w \to z \to y \to e \to J$$

Chain Rule for One Neuron

The loss depends on weights indirectly:

$$w \to z \to y \to e \to J$$

Therefore,

$$\frac{\partial J}{\partial w} = \frac{\partial J}{\partial e} \cdot \frac{\partial e}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w}$$

Chain Rule for One Neuron

The loss depends on weights indirectly:

$$w \to z \to y \to e \to J$$

Therefore,

$$\frac{\partial J}{\partial w} = \frac{\partial J}{\partial e} \cdot \frac{\partial e}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w}$$

For the bias,

$$\frac{\partial J}{\partial b} = \frac{\partial J}{\partial e} \cdot \frac{\partial e}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial b}$$

Chain Rule for One Neuron

The loss depends on weights indirectly:

$$w \to z \to y \to e \to J$$

Therefore,

$$\frac{\partial J}{\partial w} = \frac{\partial J}{\partial e} \cdot \frac{\partial e}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w}$$

For the bias,

$$\frac{\partial J}{\partial b} = \frac{\partial J}{\partial e} \cdot \frac{\partial e}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial b}$$

Important

Backpropagation is just repeated use of the chain rule.

Deriving the Delta Term

We use

$$J = \frac{1}{2}(y_d - y)^2, \qquad e = y_d - y, \qquad y = \sigma(z)$$

Deriving the Delta Term

We use

$$J = \frac{1}{2}(y_d - y)^2, \qquad e = y_d - y, \qquad y = \sigma(z)$$

Individual derivatives:

$$\frac{\partial J}{\partial e} = e = y_d - y$$

$$\frac{\partial e}{\partial y} = -1$$

$$\frac{\partial y}{\partial z} = y(1 - y)$$

Deriving the Delta Term

We use

$$J = \frac{1}{2}(y_d - y)^2, \qquad e = y_d - y, \qquad y = \sigma(z)$$

Individual derivatives:

$$\frac{\partial J}{\partial e} = e = y_d - y$$

$$\frac{\partial e}{\partial y} = -1$$

$$\frac{\partial y}{\partial z} = y(1 - y)$$

Now combine:

$$\delta = \frac{\partial J}{\partial z} = (y_d - y)(-1)y(1 - y)$$

Deriving the Delta Term

We use

$$J = \frac{1}{2}(y_d - y)^2, \qquad e = y_d - y, \qquad y = \sigma(z)$$

Individual derivatives:

$$\frac{\partial J}{\partial e} = e = y_d - y$$

$$\frac{\partial e}{\partial y} = -1$$

$$\frac{\partial y}{\partial z} = y(1 - y)$$

Now combine:

$$\delta = \frac{\partial J}{\partial z} = (y_d - y)(-1)y(1 - y)$$

$$\boxed{\delta = (y - y_d)y(1 - y)}$$

Gradients with Respect to Weights and Bias

Since

$$z = w_1 x_1 + w_2 x_2 + b$$

Gradients with Respect to Weights and Bias

Since

$$z = w_1 x_1 + w_2 x_2 + b$$

we have

$$\frac{\partial z}{\partial w_1} = x_1, \qquad \frac{\partial z}{\partial w_2} = x_2, \qquad \frac{\partial z}{\partial b} = 1$$

Gradients with Respect to Weights and Bias

Since

$$z = w_1 x_1 + w_2 x_2 + b$$

we have

$$\frac{\partial z}{\partial w_1} = x_1, \qquad \frac{\partial z}{\partial w_2} = x_2, \qquad \frac{\partial z}{\partial b} = 1$$

Therefore,

$$\boxed{\frac{\partial J}{\partial w_1} = \delta x_1}$$

$$\boxed{\frac{\partial J}{\partial w_2} = \delta x_2}$$

$$\boxed{\frac{\partial J}{\partial b} = \delta}$$

Gradients with Respect to Weights and Bias

Since

$$z = w_1 x_1 + w_2 x_2 + b$$

we have

$$\frac{\partial z}{\partial w_1} = x_1, \qquad \frac{\partial z}{\partial w_2} = x_2, \qquad \frac{\partial z}{\partial b} = 1$$

Therefore,

$$\boxed{\frac{\partial J}{\partial w_1} = \delta x_1}$$

$$\boxed{\frac{\partial J}{\partial w_2} = \delta x_2}$$

$$\boxed{\frac{\partial J}{\partial b} = \delta}$$

Meaning

A weight is updated strongly only when both the error signal and the input to that weight are active.

Gradient Descent Update Rule

Let the learning rate be

$$\eta = 1 \qquad \text{(chosen only for easy hand calculation)}$$

Gradient Descent Update Rule

Let the learning rate be

$$\eta = 1 \qquad \text{(chosen only for easy hand calculation)}$$

Weights and bias are updated as

$$w_1^{new} = w_1^{old} - \eta\delta x_1$$
$$w_2^{new} = w_2^{old} - \eta\delta x_2$$
$$b^{new} = b^{old} - \eta\delta$$

Gradient Descent Update Rule

Let the learning rate be

$$\eta = 1 \qquad \text{(chosen only for easy hand calculation)}$$

Weights and bias are updated as

$$w_1^{new} = w_1^{old} - \eta\delta x_1$$
$$w_2^{new} = w_2^{old} - \eta\delta x_2$$
$$b^{new} = b^{old} - \eta\delta$$

Forward pass $\rightarrow$ Loss $\rightarrow$ Delta $\rightarrow$ Update

Manual Training Setup for OR Gate

Training samples

$$(0,0) \to 0$$

$$(0,1) \to 1$$

$$(1,0) \to 1$$

$$(1,1) \to 1$$

Initial parameters

$$w_1 = 0, \qquad w_2 = 0, \qquad b = 0$$

$$\eta = 1$$

Important distinction

One sample update is one iteration. One complete pass through all four samples is one epoch.

Iteration 1: $(0, 0) \rightarrow 0$

Current parameters

$$w_1 = 0, \qquad w_2 = 0, \qquad b = 0$$

Iteration 1: $(0, 0) \rightarrow 0$

Current parameters

$$w_1 = 0, \qquad w_2 = 0, \qquad b = 0$$

Forward pass

$$z = 0(0) + 0(0) + 0 = 0$$

$$y = \sigma(0) = 0.5$$

Iteration 1: $(0, 0) \rightarrow 0$

Current parameters

$$w_1 = 0, \qquad w_2 = 0, \qquad b = 0$$

Forward pass

$$z = 0(0) + 0(0) + 0 = 0$$

$$y = \sigma(0) = 0.5$$

Delta

$$\delta = (0.5 - 0)(0.5)(1 - 0.5) = 0.125$$

Iteration 1: $(0, 0) \rightarrow 0$

Current parameters

$$w_1 = 0, \qquad w_2 = 0, \qquad b = 0$$

Forward pass

$$z = 0(0) + 0(0) + 0 = 0$$

$$y = \sigma(0) = 0.5$$

Delta

$$\delta = (0.5 - 0)(0.5)(1 - 0.5) = 0.125$$

Update

$$w_1 = 0 - 1(0.125)(0) = 0$$

$$w_2 = 0 - 1(0.125)(0) = 0$$

$$b = 0 - 1(0.125) = -0.125$$

Iteration 2: $(0, 1) \rightarrow 1$

Current parameters

$$w_1 = 0, \qquad w_2 = 0, \qquad b = -0.125$$

Iteration 2: $(0, 1) \rightarrow 1$

Current parameters

$$w_1 = 0, \qquad w_2 = 0, \qquad b = -0.125$$

Forward pass

$$z = 0(0) + 0(1) - 0.125 = -0.125$$

$$y = \sigma(-0.125) = 0.4688$$

Iteration 2: $(0, 1) \rightarrow 1$

Current parameters

$$w_1 = 0, \qquad w_2 = 0, \qquad b = -0.125$$

Forward pass

$$z = 0(0) + 0(1) - 0.125 = -0.125$$

$$y = \sigma(-0.125) = 0.4688$$

Delta

$$\delta = (0.4688 - 1)(0.4688)(1 - 0.4688) = -0.1323$$

Iteration 2: $(0, 1) \rightarrow 1$

Current parameters

$$w_1 = 0, \qquad w_2 = 0, \qquad b = -0.125$$

Forward pass

$$z = 0(0) + 0(1) - 0.125 = -0.125$$

$$y = \sigma(-0.125) = 0.4688$$

Delta

$$\delta = (0.4688 - 1)(0.4688)(1 - 0.4688) = -0.1323$$

Update

$$w_1 = 0 - 1(-0.1323)(0) = 0$$

$$w_2 = 0 - 1(-0.1323)(1) = 0.1323$$

$$b = -0.125 - 1(-0.1323) = 0.0073$$

Iteration 3: $(1, 0) \rightarrow 1$

Current parameters

$$w_1 = 0, \qquad w_2 = 0.1323, \qquad b = 0.0073$$

Iteration 3: $(1, 0) \rightarrow 1$

Current parameters

$$w_1 = 0, \qquad w_2 = 0.1323, \qquad b = 0.0073$$

Forward pass

$$z = 0(1) + 0.1323(0) + 0.0073 = 0.0073$$

$$y = \sigma(0.0073) = 0.5018$$

Iteration 3: $(1, 0) \rightarrow 1$

Current parameters

$$w_1 = 0, \qquad w_2 = 0.1323, \qquad b = 0.0073$$

Forward pass

$$z = 0(1) + 0.1323(0) + 0.0073 = 0.0073$$

$$y = \sigma(0.0073) = 0.5018$$

Delta

$$\delta = (0.5018 - 1)(0.5018)(1 - 0.5018) = -0.1246$$

Iteration 3: $(1, 0) \rightarrow 1$

Current parameters

$$w_1 = 0, \qquad w_2 = 0.1323, \qquad b = 0.0073$$

Forward pass

$$z = 0(1) + 0.1323(0) + 0.0073 = 0.0073$$

$$y = \sigma(0.0073) = 0.5018$$

Delta

$$\delta = (0.5018 - 1)(0.5018)(1 - 0.5018) = -0.1246$$

Update

$$w_1 = 0 - 1(-0.1246)(1) = 0.1246$$

$$w_2 = 0.1323 - 1(-0.1246)(0) = 0.1323$$

$$b = 0.0073 - 1(-0.1246) = 0.1319$$

Iteration 4: $(1, 1) \rightarrow 1$

Current parameters

$$w_1 = 0.1246, \qquad w_2 = 0.1323, \qquad b = 0.1319$$

Iteration 4: $(1, 1) \rightarrow 1$

Current parameters

$$w_1 = 0.1246, \qquad w_2 = 0.1323, \qquad b = 0.1319$$

Forward pass

$$z = 0.1246(1) + 0.1323(1) + 0.1319 = 0.3888$$

$$y = \sigma(0.3888) = 0.5960$$

Iteration 4: $(1, 1) \rightarrow 1$

Current parameters

$$w_1 = 0.1246, \qquad w_2 = 0.1323, \qquad b = 0.1319$$

Forward pass

$$z = 0.1246(1) + 0.1323(1) + 0.1319 = 0.3888$$

$$y = \sigma(0.3888) = 0.5960$$

Delta

$$\delta = (0.5960 - 1)(0.5960)(1 - 0.5960) = -0.0973$$

Iteration 4: $(1, 1) \rightarrow 1$

Current parameters

$$w_1 = 0.1246, \qquad w_2 = 0.1323, \qquad b = 0.1319$$

Forward pass

$$z = 0.1246(1) + 0.1323(1) + 0.1319 = 0.3888$$

$$y = \sigma(0.3888) = 0.5960$$

Delta

$$\delta = (0.5960 - 1)(0.5960)(1 - 0.5960) = -0.0973$$

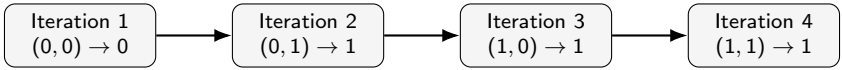
Update

$$w_1 = 0.1246 - 1(-0.0973)(1) = 0.2219$$

$$w_2 = 0.1323 - 1(-0.0973)(1) = 0.2296$$

$$b = 0.1319 - 1(-0.0973) = 0.2292$$

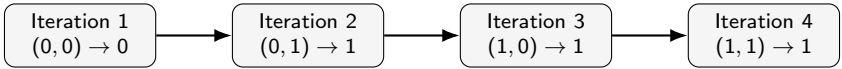
One Epoch Completed



Final parameters after one epoch:

$$w_1 = 0.2219, \qquad w_2 = 0.2296, \qquad b = 0.2292$$

One Epoch Completed

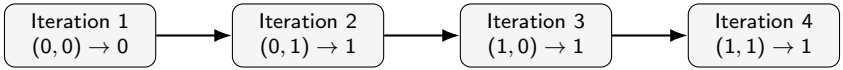


Final parameters after one epoch:

$$w_1 = 0.2219, \qquad w_2 = 0.2296, \qquad b = 0.2292$$

4 iterations = 1 epoch

One Epoch Completed



Final parameters after one epoch:

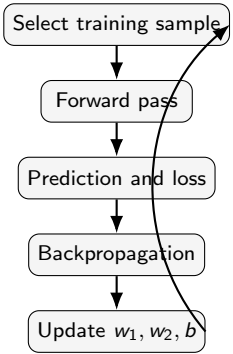
$$\boxed{w_1 = 0.2219, \qquad w_2 = 0.2296, \qquad b = 0.2292}$$

4 iterations = 1 epoch

What happens next?

Repeat the same four samples for more epochs until the loss becomes small.

Training Loop for the OR Gate



One epoch

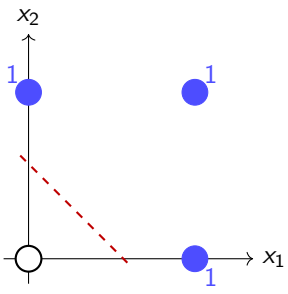
One complete pass through all four OR-gate samples.

- The parameters change after each sample.
- Repeating epochs gradually reduces total loss.
- The decision boundary moves toward a correct separator.

Training = many repeated correction steps

Why One Neuron Can Learn OR

OR data in the input plane



Linear separability

The negative sample $(0, 0)$ can be separated from all positive samples using one straight line.

A single sigmoid neuron creates a boundary of the form

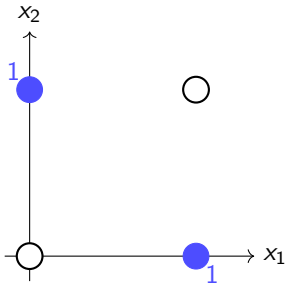
$$w_1 x_1 + w_2 x_2 + b = 0$$

Therefore, a single neuron is sufficient for OR.

The XOR Challenge: Why We Need an MLP

XOR truth table

x_1	x_2	y_d
0	0	0
0	1	1
1	0	1
1	1	0



Question

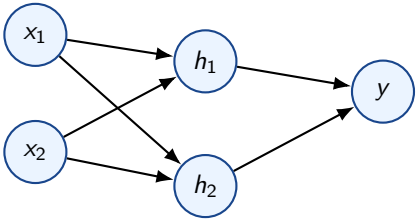
Can one straight line separate the two classes?

Conclusion

XOR is not linearly separable. A hidden layer combines multiple boundaries, allowing a multi-layer perceptron to solve the problem.

XOR With a 2–2–1 MLP

Network architecture



Forward equations

$$\mathbf{z}^{[1]} = W^{[1]}\mathbf{x} + \mathbf{b}^{[1]}$$

$$\mathbf{a}^{[1]} = \sigma\left(\mathbf{z}^{[1]}\right)$$

$$z^{[2]} = W^{[2]}\mathbf{a}^{[1]} + b^{[2]}$$

$$y = \sigma\left(z^{[2]}\right)$$

Important idea

Hidden neurons create intermediate features. The output neuron combines those features.

Exact XOR Construction Using Step Neurons

Use the step activation

$$H(s) = \begin{cases} 1, & s \geq 0 \\ 0, & s < 0 \end{cases}$$

Choose two hidden neurons:

$$h_1 = H(x_1 + x_2 - 0.5)$$

$$h_2 = H(-x_1 - x_2 + 1.5)$$

Meaning

$$h_1 = x_1 \text{ OR } x_2$$

$$h_2 = x_1 \text{ NAND } x_2$$

Output neuron:

$$y = H(h_1 + h_2 - 1.5)$$

Key identity

$$\boxed{x_1 \oplus x_2 = (x_1 \vee x_2) \wedge (x_1 \text{ NAND } x_2)}$$

Manual Verification of XOR MLP

x_1	x_2	$h_1 = H(x_1 + x_2 - 0.5)$	$h_2 = H(-x_1 - x_2 + 1.5)$	$y = H(h_1 + h_2 - 1.5)$
0	0	0	1	0
0	1	1	1	1
1	0	1	1	1
1	1	1	0	0

Conclusion

The hidden layer transforms the input space so that the output neuron can separate the classes.

XOR MLP as Weight Matrices

From the previous construction:

$$W^{[1]} = \begin{bmatrix} 1 & 1 \\ -1 & -1 \end{bmatrix}$$

$$\mathbf{b}^{[1]} = \begin{bmatrix} -0.5 \\ 1.5 \end{bmatrix}$$

Output layer:

$$W^{[2]} = \begin{bmatrix} 1 & 1 \end{bmatrix}$$

$$b^{[2]} = -1.5$$

In practice

With sigmoid neurons, the same idea is learned approximately through backpropagation.

Backpropagation Derivation for XOR MLP

For one training sample:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}, \qquad J = \frac{1}{2}(y_d - y)^2$$

Output layer error signal

$$\delta^{[2]} = \frac{\partial J}{\partial z^{[2]}} = (y - y_d)\sigma'(z^{[2]}) = (y - y_d)y(1 - y)$$

Hidden layer error signal

$$\boldsymbol{\delta}^{[1]} = \frac{\partial J}{\partial \mathbf{z}^{[1]}} = \left((W^{[2]})^T \delta^{[2]}\right) \odot \sigma'\left(\mathbf{z}^{[1]}\right)$$

$$\sigma'\left(\mathbf{z}^{[1]}\right) = \mathbf{a}^{[1]} \odot \left(1 - \mathbf{a}^{[1]}\right)$$

Gradient and Weight Updates for XOR MLP

Output layer gradients

$$\frac{\partial J}{\partial W^{[2]}} = \delta^{[2]}(\mathbf{a}^{[1]})^T$$

$$\frac{\partial J}{\partial b^{[2]}} = \delta^{[2]}$$

Hidden layer gradients

$$\frac{\partial J}{\partial W^{[1]}} = \boldsymbol{\delta}^{[1]}\mathbf{x}^T$$

$$\frac{\partial J}{\partial \mathbf{b}^{[1]}} = \boldsymbol{\delta}^{[1]}$$

Gradient descent update

$$W^{[2]} \leftarrow W^{[2]} - \eta\frac{\partial J}{\partial W^{[2]}}$$

$$b^{[2]} \leftarrow b^{[2]} - \eta\frac{\partial J}{\partial b^{[2]}}$$

$$W^{[1]} \leftarrow W^{[1]} - \eta\frac{\partial J}{\partial W^{[1]}}$$

$$\mathbf{b}^{[1]} \leftarrow \mathbf{b}^{[1]} - \eta\frac{\partial J}{\partial \mathbf{b}^{[1]}}$$

Teaching point

Each weight update contains an error signal multiplied by the activation feeding that weight.

Hand Calculation Exercise: One XOR Sample

Use the sample

$$(x_1, x_2) = (0, 1), \qquad y_d = 1$$

1. Compute hidden net inputs:

$$z_1^{[1]} = \_\_\_\_\_, \qquad z_2^{[1]} = \_\_\_\_\_$$

2. Compute hidden activations:

$$a_1^{[1]} = \_\_\_\_\_, \qquad a_2^{[1]} = \_\_\_\_\_$$

3. Compute output:

$$z^{[2]} = \_\_\_\_\_, \qquad y = \_\_\_\_\_$$

4. Compute loss and update weights using

$$W \leftarrow W - \eta \nabla J$$

This is the same learning loop as OR, but now gradients pass through two layers.

Google Colab Practical: OR-Gate Training

What You Will Implement

1. Define the OR-gate training samples.
2. Implement the sigmoid neuron and forward pass.
3. Calculate the loss and backpropagation gradients.
4. Update the weights and bias over multiple epochs.
5. Visualize the loss and verify the final predictions.

Learning Objective

Connect the manual derivation with a complete Python implementation of forward propagation, backpropagation, and gradient descent.

Scan to Run the Colab Notebook

Open the notebook, connect to the runtime, and execute each cell sequentially.

ANN BP Fundamentals Quiz

Instructions

- Scan the QR code to access the quiz.
- Sign in using your **VIT Google account**.
- The quiz contains **10 questions**.
- Only **one submission per account** is allowed.

QUIZ: Scan to begin

Summary

- A single sigmoid neuron can learn the OR gate because OR is linearly separable.
- Backpropagation is the chain rule applied from loss back to parameters.
- One iteration means one sample update.
- One epoch means one complete pass through all training samples.
- XOR is not linearly separable, so it requires a hidden layer.

OR teaches learning. XOR teaches the need for depth.